

bbtkv3: An Open-Source Suite for High-Precision Timing Measurement and Analysis with the Black Box ToolKit v3

Christophe Pallier
Cognitive Neuroimaging Unit
CNRS INSERM CEA

Experimental psychologists require sub-millisecond accuracy in measuring stimulus presentation and participant response times. The Black Box ToolKit (BBTK) is a widely used hardware solution for such measurements. One limitation is that the software provided along the BBTK by the vendor is specific for Windows. We developed a cross-platform suite of command line tools and a library designed to interface with the BBTK v3. These tools included serial port detection, sensor threshold adjustment and automated event capture. Additionally, it includes an interactive shell for manual device control and a statistical analysis tool that computes event durations, jitter, and latencies from captured data. Built in Go, the suite is distributed as self-contained binaries for Windows, macOS, and Linux. The source code and a library for embedding BBTK control in Go applications are available under the GPL-3.0 licence at <https://github.com/chrplr/bbtkv3>.

Keywords: timing accuracy, stimulus presentation, response time, Black Box ToolKit, open-source software, behavioral research methods

Introduction

Precise timing is a cornerstone of experimental psychology. Variations in stimulus presentation or response recording can introduce noise or even systematic biases that confound experimental results (Plant et al., 2004). While modern computers are capable of high performance, their multi-tasking operating systems and complex hardware architectures (e.g., display buffering, USB polling) often introduce unpredictable latencies.

The Black Box ToolKit (BBTK) was developed to address these challenges by providing an external hardware reference for timing measurements. It acts as a digital oscilloscope and signal generator, capable of capturing activity on light and sound sensors, as well as TTL signals, with sub-millisecond precision.

As shown in Figure 1, the typical experimental setup involves three components:

1. A **stimulation device** (e.g., a computer) running the experimental paradigm.
2. The **BBTK hardware** with input sensors (photodiodes, microphones) and TTL inputs/outputs connected to the stimulation device.

3. A **host computer** (which may be the same as the stimulation device with our software) driving the BBTK and recording the data.

The BBTK communicates with the host computer via a serial protocol described in the BBTK API guide Plant, 2016. The `bbtkv3` software suite that we developed encapsulates this protocol into a set of command-line tools and a high-level library. Designed to be cross-platform (while the vendor's software is specific to Windows), it is easy to install, and can be integrated into automated testing workflows. One of its advantages is that the event capture can run simultaneously on the same computer that runs the stimulation.

Software Suite Components

The `bbtkv3` suite consists of several modular CLI tools, summarized in Table 1. This modularity allows researchers to use only the components they need or to chain them together in scripts.

Core Functionality

Calibration and Monitoring

Proper calibration of sensor thresholds is critical for reliable data. The `bbtk-adjust-thresholds` and `bbtk-input-check` tools provide researchers with immediate feedback on sensor activity, allowing them to set sensitivity levels that distinguish stimuli from background noise or ambient light.

 Christophe Pallier Correspondence should be addressed to INSERM, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org



Figure 1

Principle of operation: The BBTK acts as an independent observer of the stimulation device's outputs. Here a optical sensor detects when the illumination of the screen reaches a certain threshold. Here, the 'bbtk-capture' program that drives the BBTK device during events capture runs on the stimulation computer.

Table 1

The bbtkv3 toolset

Tool	Description
bbtk-detect-port	Scans serial ports to locate the connected BBTK.
bbtk-capture	Captures events for a specified duration and exports them to CSV and binary formats.
bbtk-input-check	Provides a live stream of sensor states for real-time validation.
bbtk-adjust-thresholds	Interactive menu for sensor sensitivity calibration.
bbtk-get-thresholds	Reads current sensor thresholds from the device.
bbtk-set-thresholds	Writes new threshold values to the device.
bbtk-event-marking	Configures the device for real-time event marking.
events-stats	Performs statistical analysis on captured timing data.
ibbtk	An interactive shell providing access to all device functions.

Event Capture and Analysis

The primary uses of bbtkv3 is the objective measurement of latencies. For example, a researcher might want to measure the delay between a TTL trigger sent by the experimental software and the actual appearance of a stimulus on the screen (detected by a photodiode).

The bbtk-capture tool automates this process. It can be launched for a specific duration, during which it records all transitions on the 12 input lines (4 optical, 2 microphone, 2 TTL, and 4 keypad inputs).

```
# Detect port and capture for 60 seconds
BBTK_PORT=$(bbtk-detect-port)
bbtk-capture -d 60
events-stats bbtk-capture-001.events.csv
```

Listing 1: Example capture command

At the end of the event capture, the data is downloaded from the BBTK onto the host PC and saved for further analyses. Among the saved files, a .dat file contains the raw output of the BBTK, and an .events.csv file contains a table with three columns: type, onset, duration. type specifies the name of the sensor.

A tool is provided, events-stats, which processes the '.events.csv' file to produce descriptive statistics and ASCII histograms. It computes a) Event durations (Distribution of how long each sensor remained active); b) Inter-onset intervals (Timing regularity of successive events on a given sensor) and c) Paired-event onset differences (The latency between a reference event (e.g., TTL trigger) and a target event (e.g., photodiode detection). The listing below show the output of an event file generated by a stimulation on a Raspberry 4.

```
=== Duration Statistics (ms) ===
```

Type	N	Min	P10	P20	P30	P40	P50	P60	P70	P80	P90
	Max	Range	P99.5-P0.5	P95-P05	SD						
Opto1	1000	36.500	37.000	37.000	37.000	37.250	37.250	37.250	37.250	37.500	
	37.500	37.750	1.250	1.000	0.500	0.204					

```
histogram (10 bins):
```

```
[ 36.500, 36.625) ms : 2
[ 36.625, 36.750) ms : 0
[ 36.750, 36.875) ms : 28 **
[ 36.875, 37.000) ms : 0
[ 37.000, 37.125) ms : 283 *****
[ 37.125, 37.250) ms : 0
[ 37.250, 37.375) ms : 449 *****
[ 37.375, 37.500) ms : 0
[ 37.500, 37.625) ms : 229 *****
[ 37.625, 37.750) ms : 9
```

```
=== Inter-Onset Interval / Jitter Statistics (ms) ===
```

Type	N	Min	P10	P20	P30	P40	P50	P60	P70	P80	P90
	Max	Range	P99.5-P0.5	P95-P05	SD						
Opto1	999	49.250	49.750	49.750	50.000	50.000	50.000	50.000	50.000	50.250	
	50.250	50.500	1.250	1.000	0.500	0.191					

```
histogram (10 bins):
```

```
[ 49.250, 49.375) ms : 1
[ 49.375, 49.500) ms : 0
[ 49.500, 49.625) ms : 13 *
[ 49.625, 49.750) ms : 0
[ 49.750, 49.875) ms : 270 *****
[ 49.875, 50.000) ms : 0
[ 50.000, 50.125) ms : 481 *****
[ 50.125, 50.250) ms : 0
[ 50.250, 50.375) ms : 227 *****
[ 50.375, 50.500) ms : 7
```

```
=== Paired-Event Onset Differences: TTLin1 -> Opto1 (ms) ===
```

Type	N	Min	P10	P20	P30	P40	P50	P60	P70	P80
	P90	Max	Range	P99.5-P0.5	P95-P05	SD				
TTLin1>Opto1	1000	31.000	39.000	39.000	39.175	39.250	39.250	39.250	39.250	
	39.250	39.500	39.750	8.750	1.000	0.500	0.322			

```
histogram (10 bins):
```

```
[ 31.000, 31.875) ms : 1
[ 31.875, 32.750) ms : 0
[ 32.750, 33.625) ms : 0
[ 33.625, 34.500) ms : 0
[ 34.500, 35.375) ms : 0
[ 35.375, 36.250) ms : 0
[ 36.250, 37.125) ms : 0
[ 37.125, 38.000) ms : 0
[ 38.000, 38.875) ms : 22
[ 38.875, 39.750) ms : 977 *****
```

Listing 2

Example output of events-stats

Interactive Usages

An interactive tool, `ibbtk`, allows the user to “manually” send commands. This is useful for debugging and exploration of the device’s capabilities. It maintains a persistent connection, avoiding the overhead of re-opening the serial port for each command.

Library

Furthermore, because the entire suite is built upon a modular Go package, developers can easily integrate BBTK control directly into their own Go-based experimental or analysis software.

Implementation and Availability

`bbtkv3` is written in Go 1.25, leveraging the `go.bug.st/serial` library for robust cross-platform serial communication. By using Go, we are able to provide self-contained, statically linked binaries that require no runtime environment (like Python or Java) or external DLLs.

The software is available for Windows (x86-64 and ARM64), macOS (Intel and Apple Silicon), and Linux. Source code and pre-compiled binaries are hosted on GitHub

(<https://github.com/chrplr/bbtkv3>) under the GNU General Public Licence v3.

Author Contributions

C.P. conceptualized the software, designed the library, and implemented the tools. Claude Sonnet and Gemini 2.5 were used in the final phase of the project, to clean some bugs and to develop the event-stats tool. Consistent with current editorial norms (Springer Nature, 2023). C.P. takes full responsibility for the content of the work.

References

- Plant, R. (2016). *The black box toolkit v2. api guide. revision rc4*. The Black Box ToolKit Ltd.
- Plant, R., Hammond, N., & Turner, G. (2004). Self-validating presentation and response timing in cognitive paradigms: How and why? *Behavior Research Methods, Instruments, & Computers*, 36(2), 291–303. <https://doi.org/10.3758/BF03195575>
- Springer Nature. (2023). Authorship and ai tools. <https://www.springernature.com/gp/editorial-policies/authorship>